

LAPORAN PRAKTIKUM 5

DAN TUGAS

Machine Learning



RAFKI AHMAD PAGAMANDA

2311533016

Validation Techniques & Model Selection

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

Validation Techniques & Model Selection

A. Tujuan

Praktikum ini bertujuan untuk memahami pentingnya proses validasi dalam pembelajaran mesin, khususnya dalam mengevaluasi kinerja model. Selain itu, praktikum ini juga bertujuan untuk membandingkan metode Holdout Validation dan K-Fold Cross Validation, serta mengimplementasikan teknik tersebut menggunakan library scikit-learn.

B. Teori Singkat

Dalam pembelajaran mesin, proses evaluasi model merupakan tahap penting untuk mengetahui seberapa baik model dapat bekerja pada data baru yang belum pernah dilihat sebelumnya. Model yang hanya memiliki performa baik pada data latih belum tentu dapat memberikan hasil yang baik pada data lain, sehingga diperlukan teknik validasi untuk mengukur kemampuan generalisasi model. Salah satu metode yang paling sederhana adalah Holdout Validation, yaitu dengan membagi dataset menjadi dua bagian, yaitu data latih dan data uji. Data latih digunakan untuk melatih model, sedangkan data uji digunakan untuk mengevaluasi kinerja model. Meskipun metode ini mudah diterapkan, hasil evaluasi sangat bergantung pada cara pembagian data, sehingga terkadang kurang stabil.

Untuk mengatasi keterbatasan tersebut, digunakan metode K-Fold Cross Validation. Pada metode ini, dataset dibagi menjadi beberapa bagian yang disebut fold, kemudian proses pelatihan dan pengujian dilakukan secara bergantian hingga seluruh data pernah digunakan sebagai data uji. Dengan cara ini, hasil evaluasi menjadi lebih akurat dan tidak bergantung pada satu pembagian data saja. Selain itu, teknik ini juga membantu dalam mengurangi risiko overfitting, yaitu kondisi ketika model terlalu menyesuaikan diri dengan data latih sehingga kurang baik dalam memprediksi data baru.

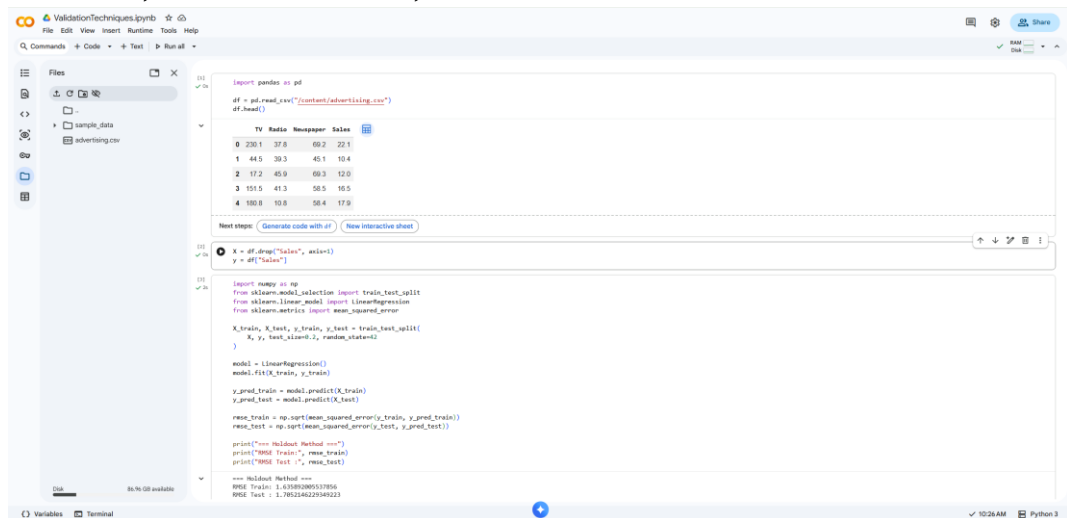
Dalam praktikum ini juga dibahas konsep model selection, yaitu proses memilih model terbaik atau konfigurasi model yang paling sesuai berdasarkan hasil evaluasi. Pemilihan model yang tepat sangat penting agar model tidak terlalu sederhana (underfitting) maupun terlalu kompleks (overfitting). Oleh karena itu, penggunaan teknik validasi seperti Holdout dan K-Fold menjadi langkah penting dalam membangun model pembelajaran mesin yang baik dan dapat diandalkan.

C. Alat dan Bahan

Alat yang digunakan dalam praktikum ini adalah laptop atau komputer dengan akses ke Google Colab atau Jupyter Notebook sebagai lingkungan pemrograman. Selain itu, digunakan bahasa pemrograman Python dengan beberapa library seperti pandas dan numpy untuk pengolahan data, serta scikit-learn untuk proses modeling dan evaluasi. Bahan yang digunakan dalam praktikum ini adalah dataset advertising.csv untuk latihan praktikum dan dataset Titanic (train.csv) untuk tugas implementasi K-Fold Cross Validation.

D. Langkah-langkah Praktikum

1. Load Data, Pemisahan Variabel, dan Holdout Validation



```
import pandas as pd
df = pd.read_csv("content/advertising.csv")
df.head()

TV Radio Newspaper Sales
0 230.1 37.8 69.2 22.1
1 44.5 39.3 45.1 10.4
2 17.2 48.9 69.3 12.8
3 101.5 41.3 58.5 16.5
4 109.8 10.8 58.4 17.9

Next steps: (Generate code with AI) (New interactive sheet)

x = df.drop("Sales", axis=1)
y = df["Sales"]

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)

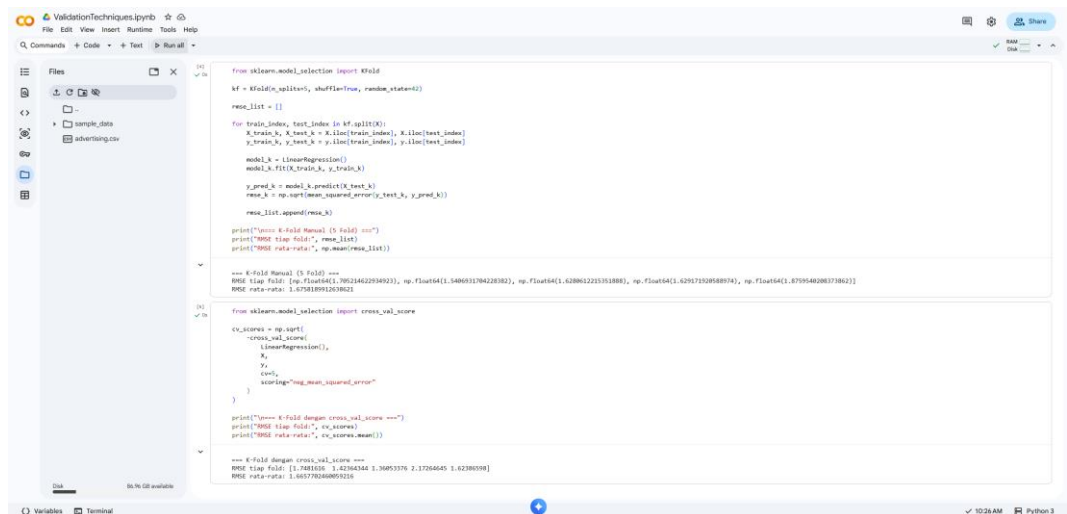
mse_train = np.mean_squared_error(y_train, y_pred_train)
mse_test = np.mean_squared_error(y_test, y_pred_test)

print("==== Holdout Method ===")
print("RMSE Train:", mse_train)
print("RMSE Test:", mse_test)

==== Holdout Method ====
RMSE Train: 1.63708308537856
RMSE Test: 1.7952146225949223
```

Pada tahap ini dilakukan proses loading dataset advertising.csv menggunakan library pandas, yang berisi data pengeluaran iklan pada media TV, Radio, dan Newspaper dengan target berupa Sales. Setelah data dimuat, dilakukan pengecekan awal menggunakan `df.head()` untuk melihat struktur data. Selanjutnya, dataset dipisahkan menjadi variabel independen (X) yang terdiri dari TV, Radio, dan Newspaper, serta variabel dependen (y) yaitu Sales. Kemudian diterapkan metode Holdout Validation dengan membagi data menjadi data latih dan data uji menggunakan `train_test_split` dengan proporsi 80% untuk data latih dan 20% untuk data uji. Model Linear Regression digunakan untuk melatih data latih, kemudian dilakukan prediksi pada data latih dan data uji. Evaluasi kinerja model dilakukan menggunakan Root Mean Squared Error (RMSE) untuk mengukur tingkat kesalahan prediksi model.

2. K-Fold Cross Validation (Manual dan cross_val_score)



```
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)

mse_list = []

for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    model_k = LinearRegression()
    model_k.fit(X_train, y_train)

    y_pred_k = model_k.predict(X_test)
    mse_k = np.mean_squared_error(y_test, y_pred_k)

    mse_list.append(mse_k)

print("==== K-fold Manual (5 fold) ===")
print("RMSE tiap fold:", mse_list)
print("RMSE rata-rata:", np.mean(mse_list))

==== K-fold Manual (5 fold) ===
RMSE tiap fold: [np.float64(1.78924422038023), np.float64(1.5486031704220382), np.float64(1.628662221351888), np.float64(1.629175108588079), np.float64(1.8709548208373822)]
RMSE rata-rata: 1.675818991263821

from sklearn.model_selection import cross_val_score

cv_scores = np.sqrt(
    -cross_val_score(
        LinearRegression(),
        X,
        y,
        cv=5,
        scoring="neg_mean_squared_error"
    )
)

print("==== K-fold dengan cross_val_score ===")
print("RMSE tiap fold:", cv_scores)
print("RMSE rata-rata:", cv_scores.mean())

==== K-fold dengan cross_val_score ===
RMSE tiap fold: [1.7481616 1.4294264 1.3085376 2.1726465 1.6238039]
RMSE rata-rata: 1.6377920808537856
```

Pada tahap ini dilakukan implementasi K-Fold Cross Validation untuk mengevaluasi kinerja model secara lebih menyeluruh. Dataset dibagi menjadi 5 bagian (fold) menggunakan KFold dengan pengacakan data. Pada metode manual, setiap fold secara bergantian digunakan sebagai data uji dan sisanya sebagai data latih, kemudian model Linear Regression dilatih dan dievaluasi menggunakan RMSE pada setiap iterasi. Nilai RMSE dari setiap fold disimpan dan dihitung rata-ratanya untuk memperoleh gambaran performa model secara keseluruhan. Selain itu, digunakan juga fungsi `cross_val_score` dari scikit-learn untuk melakukan K-Fold Cross Validation secara otomatis dengan metrik `neg_mean_squared_error` yang kemudian diubah menjadi RMSE. Hasil dari kedua metode dibandingkan untuk memastikan konsistensi dan stabilitas performa model.

TUGAS PRAKTIKUM

1. Mengerjakan semua latihan praktikum dan menyimpan file notebooknya ke dalam folder Praktikum5 pada Repo Git.

2. Lakukan analisis terhadap pengaruh Holdout Validation pada latihan praktikum
 - a. Apakah RMSE train jauh lebih kecil daripada RMSE test?

RMSE pada data train sebesar sekitar 1.64, sedangkan RMSE pada data test sebesar sekitar 1.71. Nilai RMSE pada data train memang lebih kecil dibandingkan data test, namun selisihnya ga begitu signifikan. Perbedaan yang kecil ini menunjukkan bahwa model tidak hanya bekerja baik pada data latih, tetapi juga bisa memberikan performa yang cukup baik pada data yang belum pernah dilihat sebelumnya.

- b. Apakah model overfit atau underfit?

Berdasarkan perbandingan nilai RMSE, model tidak mengalami overfitting maupun underfitting. Karena nilai RMSE pada data train dan data test relatif berdekatan, jd tidak menunjukkan adanya perbedaan performa yang mencolok. Jika model mengalami overfitting, maka RMSE pada data train akan jauh lebih kecil dibandingkan data test. Sebaliknya, jika underfitting, maka kedua nilai RMSE akan cenderung besar. Oleh karena itu, dapat disimpulkan bahwa model memiliki kemampuan generalisasi yang baik terhadap data baru.

3. Lakukan analisis thd pengaruh K-Fold Cross Validation pd latihan praktikum:
 - a. Bandingkan RMSE Holdout vs RMSE K-Fold.

Berdasarkan hasil yang di dpt kan, nilai RMSE pada metode Holdout untuk data test sekitar 1.70, sedangkan nilai RMSE rata-rata pada K-Fold berada di kisaran 1.66–1.67. Terlihat bahwa nilai RMSE pada K-Fold sedikit lebih kecil dibandingkan Holdout. Hal ini artinya K-Fold memberikan hasil evaluasi yang sedikit lebih baik. Perbedaan ini terjadi karena K-Fold menggunakan seluruh data secara bergantian sebagai data latih dan data uji, jd hasilnya lebih mewakili kondisi sebenarnya dibandingkan hanya satu kali pembagian data seperti pada Holdout.

b. Apakah K-Fold memberikan hasil yang lebih stabil?

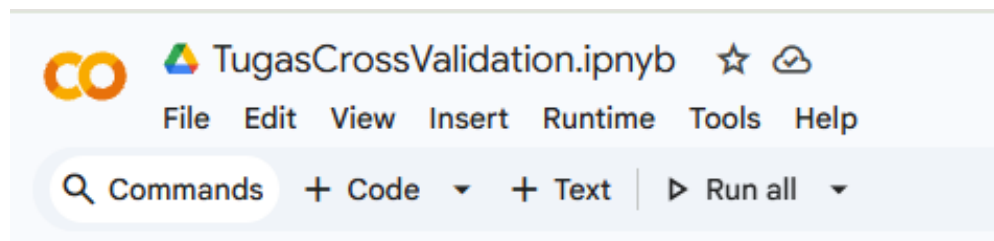
K-Fold dpt dikatakan memberikan hasil yang lebih stabil dibandingkan Holdout. Hal ini karena K-Fold melakukan proses pelatihan dan pengujian sebanyak beberapa kali dengan pembagian data yang berbeda-beda, kemudian hasilnya dirata-ratakan. Dengan cara ini, hasil evaluasi tidak terlalu dipengaruhi oleh pembagian data tertentu. Berbeda dengan Holdout yang hanya menggunakan satu kali pembagian data, sehingga hasilnya bisa berubah tergantung pembagian tersebut.

c. Bagaimana variasi kinerja model pada masing-masing fold?

Nilai RMSE pd masing-masing fold berada dalam rentang sekitar 1.5 hingga 1.8. Meskipun terdapat perbedaan nilai pada setiap fold, namun selisihnya tidak terlalu besar. Hal ini menunjukkan bahwa kinerja model cukup konsisten pada berbagai pembagian data. Variasi ini wajar terjadi karena setiap fold menggunakan data latih dan data uji yang berbeda, namun scr keseluruhan performa model tetap stabil.

4. Buat sebuah notebook baru dengan nama TugasCrossValidation.ipnyb untuk mengimplementasikan K-Fold Cross Validation untuk memvalidasi model pembelajaran yang mampu memprediksi apakah penumpang selamat atau tidak berdasarkan data yang ada. Lakukan praproses data terlebih dahulu jika diperlukan. Dataset bisa diakses di iLearn. Selanjutnya, lakukan analisis kinerja model melalui beberapa metrik evaluasi yang relevan dan jelaskan bagaimana pengaruh teknik cross validation dalam mengevaluasi kinerja model pembelajaran.

i. Membuat Notebook



Langkah pertama Adalah membuat notebook baru dgn nama spt di atas.

ii. Import Library

1. Import Library

```
[5]
✓ Os
import numpy as np
import pandas as pd

from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

Pada tahap ini dilakukan import library yang dibutuhkan, yaitu numpy dan pandas untuk pengolahan data, serta library dari scikit-learn seperti KFold dan cross_val_score untuk proses cross validation, LogisticRegression sebagai model klasifikasi, dan beberapa metrik evaluasi seperti accuracy, precision, recall, dan F1-score.

iii. Load Dataset

```
2. Load Dataset

[6]
✓ Os
df = pd.read_csv("/content/train.csv")
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Next steps: [Generate code with df](#) [New interactive sheet](#)

Pada tahap ini dataset Titanic dimuat menggunakan pandas dari file train.csv. Dataset ini berisi informasi penumpang seperti jenis kelamin, umur, kelas tiket, serta status apakah penumpang tersebut selamat atau tidak. Setelah data dimuat, dilakukan pengecekan awal menggunakan `df.head()` untuk memastikan data sudah terbaca dengan benar dan melihat struktur kolom yang tersedia.

iv. Preprocessing

```
3. Preprocessing

[10]
✓ Os
df = df.drop(["Name", "Ticket", "Cabin"], axis=1)
df["Age"].fillna(df["Age"].mean(), inplace=True)
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
df = pd.get_dummies(df, drop_first=True)

/tmp/ipykernel_1648/1802871337.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col=col, value=value, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["Age"].fillna(df["Age"].mean(), inplace=True)
/tmp/ipykernel_1648/1802871337.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col=col, value=value, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
```

Pada tahap ini dilakukan pembersihan data agar siap digunakan oleh model. Kolom yang tidak diperlukan seperti Name, Ticket, dan Cabin dihapus karena tidak terlalu berpengaruh terhadap prediksi. Selanjutnya, nilai kosong pada kolom Age diisi dengan rata-rata, dan kolom Embarked diisi dengan nilai yang paling sering muncul. Data kategorikal kemudian diubah menjadi numerik menggunakan encoding. Setelah preprocessing, data menjadi lebih bersih dan siap untuk proses modeling.

v. Memisahkan Variable

```
4. Pisahkan X dan y

[14]
✓ Os
X = df.drop("Survived", axis=1)
y = df["Survived"]
```

Pada tahap ini data dipisahkan menjadi variabel input (X) dan target (y). Variabel X berisi seluruh fitur yang digunakan untuk prediksi, sedangkan y berisi kolom Survived yang menunjukkan apakah penumpang selamat atau tidak. Pemisahan ini dilakukan agar model dapat belajar dari data input untuk memprediksi target.

vi.K-Fold Cross Validation (Accuracy)

5. K-Fold Cross Validation (ACCURACY)

```
[15]
✓ 0s kf = KFold(n_splits=5, shuffle=True, random_state=42)

model = LogisticRegression(max_iter=1000)

acc_scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')

print("Accuracy tiap fold:", acc_scores)
print("Rata-rata accuracy:", acc_scores.mean())
print("Standar deviasi:", acc_scores.std())

... Accuracy tiap fold: [0.80446927 0.78651685 0.83707865 0.76966292 0.78089888]
Rata-rata accuracy: 0.7957253154227606
Standar deviasi: 0.02353900027200346
```

Pada tahap ini dilakukan evaluasi model menggunakan K-Fold Cross Validation dengan 5 fold. Model Logistic Regression dilatih dan diuji secara bergantian pada setiap fold. Hasil yang diperoleh menunjukkan bahwa nilai accuracy pada tiap fold berada di kisaran sekitar 0.76 hingga 0.83, dengan rata-rata sekitar 0.79. Nilai ini menunjukkan bahwa model memiliki performa yang cukup baik. Selain itu, nilai standar deviasi yang kecil menunjukkan bahwa hasilnya cukup stabil dan tidak terlalu berbeda antar foldnya.

vii.Evaluasi dengan Metrik Tambahan

6. Contoh METRIK LAIN

```
[16]
✓ 2s precision = cross_val_score(model, X, y, cv=kf, scoring='precision')
recall = cross_val_score(model, X, y, cv=kf, scoring='recall')
f1 = cross_val_score(model, X, y, cv=kf, scoring='f1')

print("\nPrecision rata-rata:", precision.mean())
print("Recall rata-rata:", recall.mean())
print("F1-score rata-rata:", f1.mean())

... Precision rata-rata: 0.7479273591192868
Recall rata-rata: 0.703179292134516
F1-score rata-rata: 0.723985413334771
```

Pada tahap ini dilakukan evaluasi tambahan menggunakan precision, recall, dan F1-score. Hasil menunjukkan bahwa precision sekitar 0.75, recall sekitar 0.70, dan F1-score sekitar 0.72. Ini berarti model cukup baik dalam memprediksi, meskipun masih ada sedikit ketidakseimbangan antara kemampuan mendeteksi data positif dan negatif. Secara keseluruhan, model sudah cukup baik dan hasilnya konsisten.

viii.Analisis

Model Logistic Regression yg digunakan dalam percobaan ini mampu memberikan hasil yg cukup baik dalam memprediksi apakah penumpang selamat atau tidak. Hal ini terlihat dari nilai accuracy rata-rata yg mencapai sekitar 0.79, yg berarti model dapat memprediksi dengan benar sekitar 79% dari total data. Selain itu, nilai standar deviasi yg kecil menunjukkan bahwa hasil pada setiap fold tdk berbeda jauh, shg performa model dapat dikatakan cukup stabil dan konsisten. Jika dilihat dari metrik tambahan, nilai precision sekitar 0.75 menunjukkan bahwa sebagian besar prediksi positif yg dihasilkan model sudah tepat, sedangkan nilai recall sekitar 0.70 menunjukkan bahwa model cukup mampu mendeteksi penumpang yg benar-benar selamat, meskipun masih ada beberapa yg terlewat. Nilai F1-score yg berada di sekitar

0.72 menunjukkan keseimbangan antara precision dan recall. Penggunaan K-Fold Cross Validation dalam proses ini sangat membantu krn data digunakan secara bergantian sebagai data latih dan data uji, shg hasil evaluasi menjadi lebih akurat dan tdk bergantung pada satu pembagian data saja. Secara keseluruhan, model memiliki performa yg cukup baik dan stabil, serta hasil evaluasi yg diperoleh dapat dipercaya.

E. Kesimpulan Praktikum dan Tugas

Secara keseluruhan, praktikum ini membantu memahami bagaimana cara mengevaluasi model menggunakan metode Holdout dan K-Fold Cross Validation. Dari hasil yang diperoleh, metode Holdout cukup sederhana dan mudah digunakan, namun hasilnya bisa bergantung pada pembagian data. Sementara itu, K-Fold Cross Validation memberikan hasil yang lebih stabil dan lebih representatif karena data digunakan secara bergantian sebagai data latih dan data uji. Pada penerapan menggunakan dataset Titanic, model Logistic Regression mampu memberikan hasil yang cukup baik dengan nilai accuracy sekitar 0.79, serta didukung oleh metrik lain seperti precision, recall, dan F1-score yang menunjukkan performa model cukup seimbang. Selain itu, penggunaan cross validation terbukti membantu dalam mendapatkan evaluasi model yang lebih akurat dan tidak bias terhadap satu pembagian data saja. Dengan demikian, dapat disimpulkan bahwa teknik validation sangat penting dalam memastikan model yang dibuat memiliki performa yang baik dan mampu melakukan generalisasi terhadap data baru.

Link Repo Github :

https://github.com/akuntugasrapon/RafkiAhmadPagamanda_2311533016_ML2526.git